Segmentación semántica de escenas usando arquitectura U-Net entrenada desde cero

Oswaldo Isaias Hernández Santes
Instituto Tecnológico y de Estudios Superiores de Monterrey
Escuela de Ingeniería y Ciencias
Santiago de Querétaro, Querétaro; México
a01199004@tec.mx

***Abstract*—Este proyecto implementa un modelo de aprendizaje profundo para la segmentación semántica de escenas, es decir, asignar a cada píxel de una imagen una de 150 clases como pared, cielo, árbol, persona o carro. Utilizando el conjunto de datos ADE20K en su versión del MIT Scene Parsing Benchmark se entrenó desde cero una arquitectura U-Net con el framework TensorFlow/Keras y se evaluó con conjuntos de entrenamiento, validación y prueba de 18,189, 2,021 y 2,000 imágenes respectivamente monitoreando la función de costo (entropía cruzada), la exactitud por píxel y el mean Intersection over Union (mIoU) a lo largo de las épocas. El análisis exploratorio mostró un fuerte desbalance de clases donde las cinco clases más frecuentes ocupan cerca del 46% de los píxeles y la mitad de las clases no llega al 0.1%. El modelo base alcanzó un mIoU de 0.144 tanto en validación como en prueba con una exactitud por píxel de 0.621 en prueba mientras que en entrenamiento llegó a un mIoU de 0.420, esta diferencia junto con el aumento de la loss de validación a partir de la época 18 indican que el modelo presenta sobreajuste (overfitting) es decir, varianza alta. Como primer experimento de mejora se aplicó data augmentation con un volteo horizontal aleatorio, lo que subió el mIoU de validación a 0.160 y redujo la diferencia entre entrenamiento y validación. El modelo genera predicciones desde la consola sobre fotos nuevas y se planean experimentos con dropout, regularización L2 y pesos por clase cambiando una sola cosa a la vez para poder explicar el efecto de cada técnica.**

Keywords*—Deep Learning, Segmentación Semántica, Redes Neuronales Convolucionales (CNN), U-Net, Encoder-Decoder, Skip Connections, Conv2DTranspose, Batch Normalization, ADE20K, TensorFlow, Keras, Épocas/Epochs, Batch Size, Learning Rate, Adam, Entropía Cruzada, Exactitud por Píxel, Intersection over Union (IoU), mIoU, Desbalance de Clases, Sesgo, Varianza, Underfitting/Overfitting, Data Augmentation, Precisión Mixta, GPU

I. Introducción

Dentro de la visión por computadora existen distintas tareas que van creciendo en nivel de detalle, en la clasificación de imágenes el modelo asigna una sola etiqueta a toda la imagen, en la detección de objetos se ubica cada objeto con una caja delimitadora y en la segmentación semántica se asigna una clase a cada píxel de la imagen [1]. Este proyecto se enfoca en la segmentación semántica de escenas completas, es decir, que no solo se busca separar un objeto del fondo sino identificar todo lo que aparece en la foto desde el piso y las paredes hasta objetos pequeños como una lámpara o un reloj.

A. Contexto general

Las redes neuronales convolucionales (CNN) son la base de la mayoría de los modelos actuales de visión por computadora ya que las capas de convolución aprenden filtros que detectan patrones como bordes, texturas y formas y al apilarse permiten reconocer objetos completos.

Para la segmentación semántica no basta con una CNN de clasificación porque esta va reduciendo el tamaño de la imagen hasta quedarse con una sola predicción por lo que se necesita una arquitectura que después de extraer las características regrese al tamaño original de la imagen para dar una predicción por píxel [2], [3]. La arquitectura U-Net es una de ellas y fue propuesta originalmente para imágenes médicas [3], pero se ha vuelto un estándar para tareas de segmentación en general y es la que se utilizó en este proyecto.

B. Dataset

El conjunto de datos utilizado es ADE20K en su versión del MIT Scene Parsing Benchmark (SceneParse150) publicado por el MIT CSAIL [4] que contiene 20,210 imágenes de entrenamiento y 2,000 de validación con sus máscaras de segmentación, donde cada píxel de la máscara tiene un valor de 0 a 150, los valores del 1 al 150 corresponden a las clases (pared, edificio, cielo, piso, árbol, etc.) y el valor 0 corresponde a otros objetos que no pertenecen a ninguna de las 150 clases y que el benchmark oficial no toma en cuenta en la evaluación [5].

Las imágenes son escenas de interiores y exteriores de distintos tamaños y las anotaciones del conjunto de prueba oficial no son públicas, el dataset se publica únicamente para uso no comercial, de investigación y educativo por lo que no se incluye en el repositorio del proyecto y se descarga desde el código.

C. Objetivo

El objetivo de este proyecto es entrenar desde cero una U-Net que reciba una foto y regrese su máscara de segmentación, diagnosticar su desempeño inicial y aplicar técnicas de mejora como regularización, data augmentation, ajuste de hiperparámetros y cambios de arquitectura, cambiando una sola cosa en cada experimento con la misma división de datos, las mismas métricas y la misma semilla para poder explicar qué funcionó y por qué, solo en caso de que el modelo desde cero deje de mejorar se usará la técnica de transfer learning.

II. Análisis exploratorio de datos

A. Verificación de archivos

Antes de entrenar el modelo se revisó que cada imagen tuviera su máscara con el mismo nombre y del mismo tamaño, se encontraron las 20,210 imágenes de entrenamiento y las

2,000 de validación completas sin ninguna máscara con un tamaño distinto al de su imagen y ningún valor mayor a 150 y no fue necesario eliminar ni corregir datos, también se encontró que 4 imágenes del conjunto de entrenamiento están en escala de grises y las demás en RGB, esto es importante ya que el modelo espera siempre 3 canales de entrada.

B. Tamaño de las imágenes

Las imágenes no tienen un tamaño fijo ya que en el conjunto de entrenamiento el ancho va de 130 a 2,100 píxeles con una mediana de 512 y el alto va de 96 a 2,100 píxeles con una mediana de 480, debido a esto y a la memoria disponible en la GPU todas las imágenes se redimensionaron a 256×256 píxeles y quiere decir que la mayoría de las imágenes se reducen y que las que no son cuadradas se deforman un poco, pero se conserva la escena completa.

C. Distribución de clases

Se calculó el porcentaje de píxeles que ocupa cada clase y se encontró un desbalance muy marcado (Fig. 1) donde las cinco clases más frecuentes que son pared, edificio, cielo, piso y árbol ocupan aproximadamente el 45.6% de todos los píxeles del conjunto de entrenamiento mientras que 75 de las 150 clases ocupan menos del 0.1% cada una.

La clase más frecuente que es pared tiene aproximadamente 827 veces más píxeles que la menos frecuente que es bandera como se muestra en la Fig. 2.

Hay objetos que aparecen seguido pero que son pequeños por ejemplo la clase de lámparas (light) aparece en el 12.4% de las imágenes pero ocupa solo el 0.08% de los píxeles y al reducir las imágenes a 256×256 estos objetos quedan de unos cuantos píxeles.

La distribución de clases del conjunto de validación oficial es casi igual a la de entrenamiento con una correlación de aproximadamente 0.94 entre los porcentajes de ambos conjuntos en escala logarítmica.

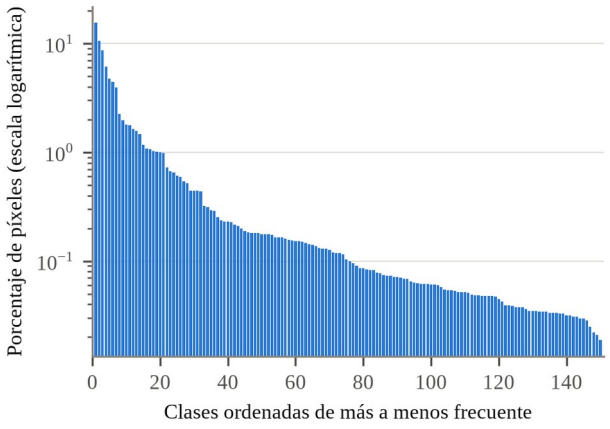


Fig. 1. Porcentaje de píxeles de las 150 clases ordenadas de más a menos frecuente en el conjunto de entrenamiento.

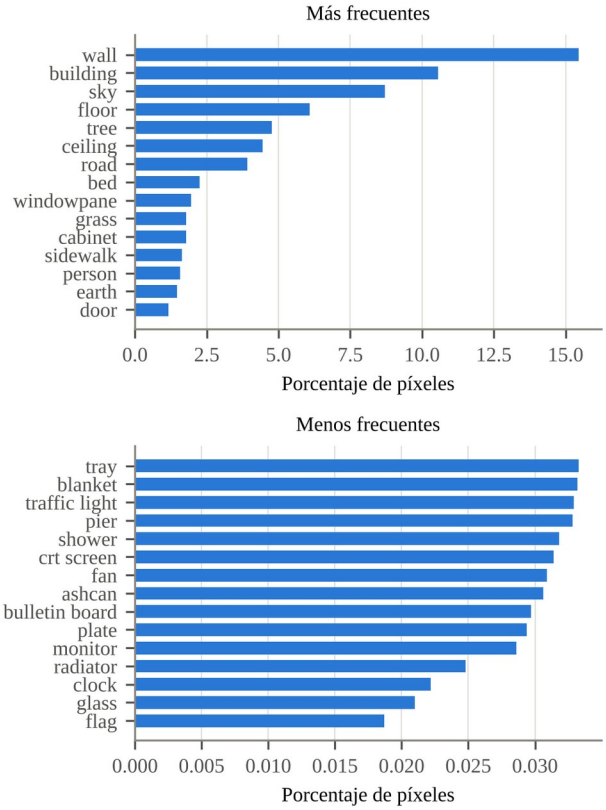


Fig. 2. Las 15 clases con más píxeles y las 15 con menos píxeles en el conjunto de entrenamiento.

D. Clase 0 (sin etiqueta)

Los píxeles con valor 0 representan el 8.64% del conjunto de entrenamiento y el 8.11% del de validación, como esta clase agrupa objetos que no tienen relación entre sí y el benchmark oficial no la evalúa [5] se decidió ignorarla en la función de costo y en las métricas, de esta forma el modelo solo aprende las 150 clases y los resultados se pueden comparar con la forma de evaluación oficial.

III. Preparación de los datos

A. División de los datos

Como las anotaciones del conjunto de prueba oficial no son públicas, el conjunto de entrenamiento oficial se dividió en entrenamiento (90%) y validación (10%) permutando aleatoriamente la lista de imágenes con una semilla fija de 42, y las 2,000 imágenes de validación oficiales se usaron como conjunto de prueba (Tabla I).

El conjunto de validación se usa para monitorear el modelo durante el entrenamiento y elegir la mejor época, mientras que el conjunto de prueba se reserva para la evaluación final [6].

La lista de qué imagen quedó en cada conjunto se guardó en un archivo CSV para que la división sea reproducible.

TABLE I. DIVISIÓN DE LOS DATOS

Conjunto	Origen	Imágenes
Entrenamiento	90% del entrenamiento oficial	18,189
Validación	10% del entrenamiento oficial	2,021
Prueba	Validación oficial	2,000

B. *Pipeline de datos*

Las imágenes se cargan con la API tf.data de TensorFlow [7] en un pipeline, primero se lee cada imagen siempre con 3 canales para que las imágenes en escala de grises queden como RGB, después la imagen se redimensiona a 256×256 con interpolación bilineal y la máscara con vecino más cercano, esto último es importante porque con otra interpolación un píxel podría quedar con un valor intermedio que no corresponde a ninguna clase.

Luego la imagen se normaliza al rango [0, 1] dividiendo entre 255 y las clases de la máscara se recorren de 1-150 a 0-149, para ignorar la clase 0 se creó un peso por píxel donde los píxeles de la clase 0 tienen un peso de 0 y el resto un peso de 1, Keras pasa estos pesos a la función de costo y a las métricas por lo que esos píxeles no cuentan en ninguna de las dos.

Finalmente las imágenes de entrenamiento se revuelven en cada época y se agrupan en batches de 8.

Para que la GPU no tenga que esperar a que se preparen las imágenes, las imágenes ya redimensionadas se guardan en memoria RAM en forma de caché después de la primera época y mientras la GPU entrena con un batch la CPU prepara el siguiente, con esto el pipeline prepara aproximadamente 2,300 imágenes por segundo sin caché y 6,900 con caché, que es mucho más de lo que procesa la GPU, usando como máximo 6.1 GB de RAM.

IV. MODELO

A. *Justificación*

Se utilizó una U-Net [3] ya que es una arquitectura de tipo codificador-decodificador (encoder-decoder) diseñada específicamente para segmentación, el codificador funciona como una CNN de clasificación que va reduciendo el tamaño de la imagen y aumentando el número de filtros para aprender qué hay en la imagen y el decodificador regresa la imagen a su tamaño original para decir dónde está cada cosa.

La característica principal de la U-Net son las conexiones de salto (skip connections) que concatenan la salida de cada nivel del codificador con el nivel correspondiente del decodificador y de esta forma se recuperan los detalles finos como los bordes de los objetos que se pierden al reducir la imagen [3].

B. *Arquitectura*

El codificador tiene 4 niveles y en cada uno se aplican dos convoluciones de 3×3 cada una seguida de batch normalization y una función de activación ReLU, después se reduce la imagen a la mitad con MaxPooling2D, el número de filtros empieza en 32 y se duplica en cada nivel hasta llegar a 512 filtros en el nivel más profundo con una imagen de 16×16 (Tabla II).

Batch normalization normaliza la salida de cada capa durante el entrenamiento lo que estabiliza y acelera el aprendizaje del modelo [8], el decodificador usa bloques upsample formados por una convolución transpuesta (Conv2DTranspose) de 3×3 con paso (stride) de 2 que duplica el tamaño de la imagen, batch normalization y ReLU y después se concatena con la skip connection del nivel correspondiente y se aplican dos convoluciones de 3×3 para combinar ambas informaciones como en la U-Net original [3].

La última capa es una convolución de 1×1 con 150 filtros que da una salida por clase en cada píxel y la clase predicha es la de mayor valor. El modelo tiene 8,645,686 parámetros.

TABLE II. ARQUITECTURA DE LA U-NET

Nivel	Tamaño de la imagen	Filtros
1	256×256	32
2	128×128	64
3	64×64	128
4	32×32	256
Más profundo	16×16	512
Salida	256×256	150 (una por clase)

C. *Función de costo y métricas*

Como función de costo se utilizó la entropía cruzada categórica por píxel (SparseCategoricalCrossentropy) ponderada con el peso de cada píxel:

$$L = -(1/N)\ \Sigma\ w_i \cdot \log(p_i) \qquad (1)$$

donde N es el número de píxeles, w_i es el peso del píxel i (0 para la clase 0 y 1 para el resto) y p_i es la probabilidad que el modelo le asigna a la clase correcta de ese píxel, como N incluye también a los píxeles con peso 0 el valor de la loss queda aproximadamente un 9% más bajo que si se excluyeran, pero esto es igual para todos los experimentos por lo que no afecta las comparaciones.

Para medir el desempeño se usaron dos métricas sin contar los píxeles de la clase 0 una es la exactitud por píxel que es el porcentaje de píxeles bien clasificados, pero en un dataset desbalanceado esta métrica puede ser engañosa ya que un modelo que solo aprenda las clases grandes como pared o cielo puede tener una exactitud alta, por esa razón la métrica principal del proyecto es el mIoU [10] donde para cada clase c se calcula la intersección sobre la unión entre los píxeles predichos y los reales:

$$IoU(c) = TP / (TP + FP + FN) \qquad (2)$$

donde TP son los píxeles de la clase c bien predichos, FP los píxeles que el modelo predijo como c pero son de otra clase y FN los píxeles de c que el modelo predijo como otra clase, el mIoU es el promedio del IoU de todas las clases, de esta forma cada clase pesa lo mismo sin importar cuántos píxeles ocupe.

D. *Entorno de entrenamiento*

El modelo se entrenó en una laptop con una GPU NVIDIA RTX 3050 Laptop de 4 GB de memoria, 16 GB de RAM,

Fedora Linux, Python 3.13 y TensorFlow 2.21, con su configuración por defecto TensorFlow solo logró reservar 2,176 MB de la GPU mientras que con el administrador de memoria cuda_malloc_async llegó a 3,648 MB, por lo que se usó este último.

Antes de entrenar se midió el tiempo por paso de una U-Net de prueba con distintos tamaños de batch, número de filtros y precisión numérica usando precisión mixta (mixed_float16) [11] que hace los cálculos en 16 bits donde es seguro, con 32 filtros y un batch de 8 se procesaron aproximadamente 75 imágenes por segundo contra 40 en 32 bits y con 64 filtros o con un batch de 16 en 32 bits el modelo ya no cabe en la memoria, entonces se eligió precisión mixta con 32 filtros iniciales y un batch de 8.

E. Hiperparámetros iniciales

El modelo base (v1) se entrenó durante 50 épocas con el optimizador Adam [12] y una tasa de aprendizaje constante de 0.001, batch de 8 y semilla de 42 para que los pesos iniciales y el orden de los datos sean reproducibles, al terminar cada época se evalúa el conjunto de validación y con un callback ModelCheckpoint se guarda el modelo únicamente cuando mejora el mIoU de validación de esta forma el modelo final corresponde a la mejor época aunque el modelo se sobreajuste después, cada época tardó aproximadamente 289 segundos y el entrenamiento completo aproximadamente 4 horas.

V. Resultados del modelo base (v1)

La Fig. 3 muestra las curvas de aprendizaje del modelo base que durante las primeras 18 épocas la loss de entrenamiento y la de validación bajan juntas, pero a partir de la época 18 donde la loss de validación llega a su mínimo de 1.334 las curvas se separan y la loss de entrenamiento sigue bajando hasta 0.392 en la época 50 mientras que la de validación sube hasta 1.921.

Lo mismo pasa con el mIoU donde el de entrenamiento sigue subiendo hasta 0.552 en la época 50 y el de validación se queda prácticamente estancado entre 0.13 y 0.14 desde la época 30.

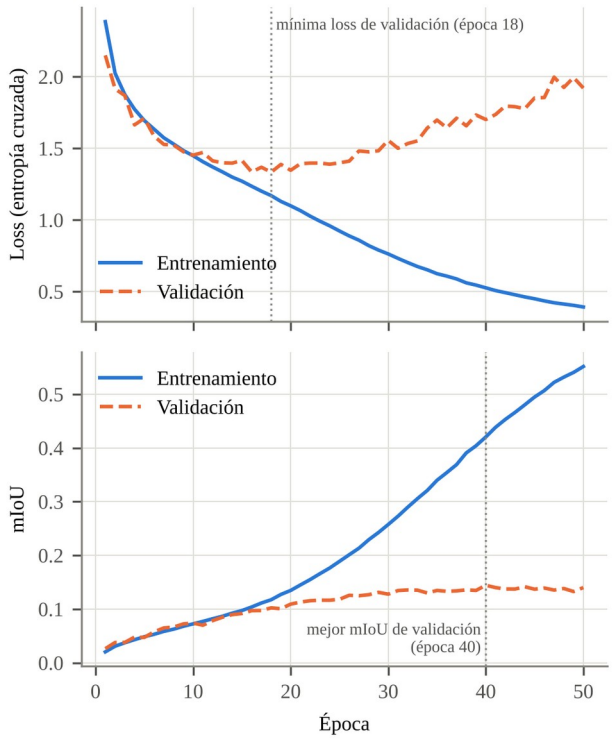


Fig. 3. Curvas de aprendizaje del modelo base (v1): loss y mIoU de entrenamiento y validación por época.

La mejor época según el mIoU de validación fue la 40 (Tabla III) con un mIoU de 0.144 en validación contra 0.420 en entrenamiento y una exactitud por píxel de 0.626 contra 0.836.

La gran diferencia entre entrenamiento y validación (varianza alta) junto con que el modelo sí logra aprender el conjunto de entrenamiento indican que el modelo presenta sobreajuste (overfitting) [13], que en otras palabras quiere decir que el modelo está memorizando las imágenes de entrenamiento en lugar de aprender patrones que le sirvan para imágenes nuevas, también se observa que entre la época 18 y la 40 la loss de validación empeora, pero el mIoU de validación todavía mejora de 0.102 a 0.144 esto se puede explicar porque el modelo predice bien en más píxeles, pero se equivoca con mucha seguridad en otros y la entropía cruzada castiga mucho los errores con alta confianza, por esta razón el mejor modelo se eligió por mIoU y no por loss.

La diferencia entre la exactitud por píxel (0.626) y el mIoU (0.144) en validación confirma lo que se esperaba del análisis exploratorio donde el modelo acierta en las clases grandes como pared, cielo o piso que dominan la exactitud pero falla en la mayoría de las 150 clases.

TABLE III. Resultados del modelo base en la mejor época

Métrica	Entrenamiento	Validación	Prueba
Loss	0.526	1.701	1.742
Exactitud por píxel	0.836	0.626	0.621
mIoU	0.420	0.144	0.144

A. *Resultados por clase*

Al evaluar el modelo base en el conjunto de prueba se obtuvo una loss de 1.742, una exactitud por píxel de 0.621 y un mIoU de 0.144 que son valores prácticamente iguales a los de validación (Tabla III) y esto quiere decir que el conjunto de validación representa bien al de prueba y que elegir la mejor época con validación no sesgó los resultados.

Al calcular el IoU de cada clase en prueba se encontró que solo 10 de las 150 clases superan un IoU de 0.5 siendo cielo (0.89), techo (0.65), calle (0.62), edificio (0.61) y árbol (0.59) las mejores mientras que 9 clases tienen un IoU de 0 como escalón, lámpara de pared (sconce), semáforo, regadera, monitor, vaso y reloj que son objetos pequeños y poco frecuentes, la clase pared aunque es la clasemás frecuente tiene un IoU de 0.54 que es menor al del cielo posiblemente porque se confunde con clases parecidas como edificio o techo.

La Fig. 4 muestra que existe una relación clara entre la frecuencia de cada clase y su IoU con una correlación de rangos de 0.71, es decir que las clases que ocupan más píxeles en entrenamiento son las que el modelo segmenta mejor y la mayoría de las clases con menos del 0.1% de los píxeles quedan con un IoU cercano a 0 esto confirma que el desbalance de clases encontrado en el análisis exploratorio es una de las causas principales del mIoU bajo.

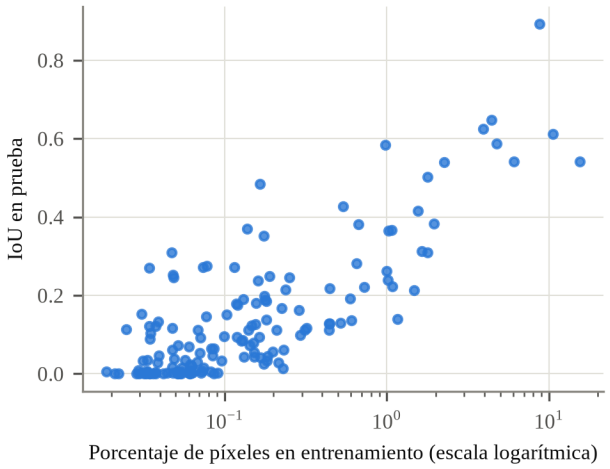


Fig. 4. IoU de cada clase del modelo base en el conjunto de prueba contra el porcentaje de píxeles que ocupa en entrenamiento.

VI. Experimento v2: data augmentation

El primer experimento de mejora cambia una sola cosa respecto al modelo base, se agregó data augmentation con un volteo horizontal aleatorio que se aplica en la mitad de los casos usando la misma decisión aleatoria para la imagen, su máscara y sus pesos para que la máscara siga correspondiendo a la imagen, se eligió esta técnica porque el volteo horizontal no cambia el significado de una escena, una calle o un cuarto volteados en espejo siguen siendo escenas válidas, de esta forma en cada época el modelo ve versiones distintas de las mismas imágenes y le es más difícil memorizarlas, el volteo solo se aplica al conjunto de entrenamiento y todo lo demás (arquitectura, hiperparámetros, división de datos y semilla) se mantuvo igual.

Como se muestra en la Tabla IV el mIoU de validación subió de 0.144 a 0.160 que es aproximadamente un 10.5% más y la exactitud por píxel de validación pasó de 0.626 a 0.649 mientras que el loss de validación bajó de 1.701 a 1.411.

Al mismo tiempo el mIoU de entrenamiento bajó de 0.420 a 0.336, esto quiere decir que el modelo memoriza menos el conjunto de entrenamiento y la diferencia entre entrenamiento y validación se redujo de 0.276 a 0.176 por lo que el data augmentation sí redujo el sobreajuste aunque no lo eliminó.

La mejor época de v2 fue la última (50) a diferencia de v1 donde fue la 40 esto puede indicar que el modelo todavía podía seguir mejorando con más épocas, en la relación entre la frecuencia de cada clase y su IoU (Fig. 5) se mantiene la misma tendencia que en v1, pero varias clases poco frecuentes alcanzan valores de IoU más altos.

TABLE IV. Comparación entre el modelo base (v1) y v2

Métrica	v1	v2
Mejor época	40	50
Loss de entrenamiento	0.526	0.672
Loss de validación	1.701	1.411
Exactitud de validación	0.626	0.649
mIoU de entrenamiento	0.420	0.336
mIoU de validación	0.144	0.160
Loss de prueba	1.742	1.444
Exactitud de prueba	0.621	0.644
mIoU de prueba	0.144	0.166

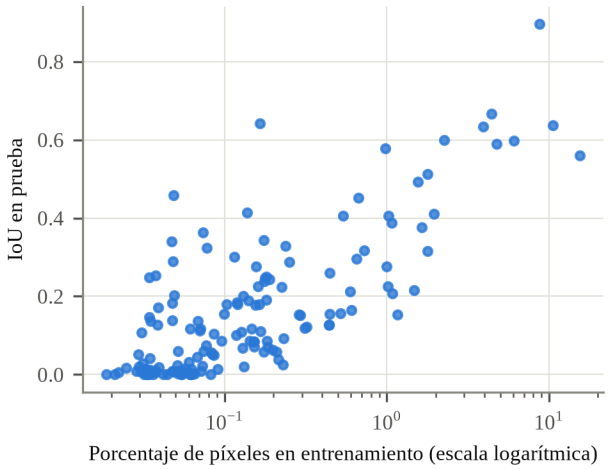


Fig. 5. IoU de cada clase del modelo v2 en el conjunto de prueba contra el porcentaje de píxeles que ocupa en entrenamiento.

VII. Predicciones

El modelo puede generar predicciones desde la consola con el script predict.py que recibe una o varias fotos, las prepara igual que en el entrenamiento, predice la clase de cada píxel, regresa la máscara al tamaño original de la foto y guarda una figura con la foto original, la máscara predicha, la máscara sobre la foto y la máscara real cuando la foto pertenece al dataset además de impreme las clases detectadas con el porcentaje de la imagen que ocupa cada una.

Para probar el modelo con imágenes que no pertenecen al dataset se tomaron 7 fotos propias con un celular en exteriores (estacionamientos, plazas y jardines) y se segmentaron con los modelos v1 y v2.

Como estas fotos no tienen máscara real no se pueden calcular métricas, por lo que la evaluación es visual, pero en general ambos modelos reconocen bien las regiones grandes de la escena como el cielo, los árboles, los edificios y el suelo (Fig. 6) aunque tienen problemas con los objetos pequeños y con superficies parecidas entre sí, por ejemplo el piso de las plazas lo confunden entre tierra (earth), banqueta (sidewalk), camino (path) y piso (floor) y en algunas fotos aparecen en regiones pequeñas clases que no están en la escena como barco, fuente, agua o avión, también se observó que los carros de un estacionamiento fotografiado a contraluz no se reconocieron como carro.

Al comparar ambos modelos el v2 genera regiones más uniformes, por ejemplo en la primera foto de la Fig. 6 el v1 divide el piso de la plaza en varias clases (banqueta, camino, pista de tierra y calle) mientras que el v2 lo marca casi completo como banqueta y en la segunda foto el v2 delimita mejor las jardineras con flores (plant).

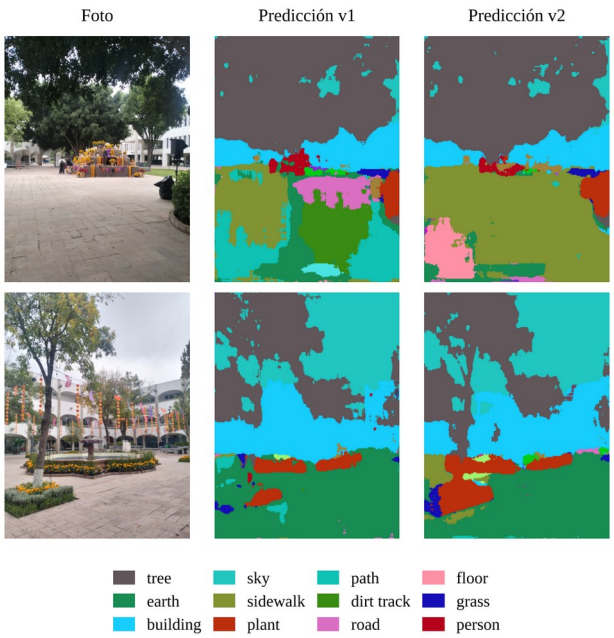


Fig. 6. Predicciones de los modelos v1 y v2 sobre dos fotos propias tomadas en el campus.

VIII. Trabajo futuro

Como el modelo sigue presentando sobreajuste después de usar data augmentation los siguientes experimentos van a seguir enfocados en reducir la varianza agregando dropout en el decodificador y regularización L2 en los pesos, también se probarán pesos por clase en la función de costo para darle más importancia a las clases poco frecuentes ya que el análisis por clase mostró que son las que más bajan el mIoU y entrenar durante más épocas ya que la mejor época de v2 fue la última, cada experimento cambiando una sola cosa respecto al anterior.

Además se puede aplicar transfer learning con un codificador preentrenado en caso de que el modelo desde cero deje de mejorar y también se puede mejorar la interfaz actual hecha con Streamlit para que tenga un diseño tipo chat.

IX. Conclusiones

En este proyecto se implementó y entrenó desde cero una U-Net para la segmentación semántica de escenas con el dataset ADE20K usando TensorFlow/Keras.

El modelo base alcanzó un mIoU de 0.144 en validación y en prueba con una exactitud por píxel de 0.621 en prueba, valores bajos pero esperables para un modelo entrenado desde cero con imágenes de 256×256, 150 clases muy desbalanceadas y los recursos limitados de una GPU de 4 GB.

Los resultados muestran de forma clara que el problema principal del modelo es el sobreajuste y no la falta de capacidad ya que en el entrenamiento el modelo sigue mejorando mientras que en validación se estanca, usar data augmentation con volteo horizontal confirmó este diagnóstico ya que al hacer más difícil que el modelo memorice las imágenes el mIoU de validación subió a 0.160 y se redujo la diferencia con entrenamiento, también se comprobó que en un problema con clases desbalanceadas la exactitud por píxel puede dar una idea equivocada del desempeño del modelo y que el mIoU refleja mejor qué tan bien se segmentan todas las clases además de que las clases poco frecuentes son las que el modelo segmenta peor.

En general el proyecto muestra que una U-Net entrenada desde cero con recursos limitados puede aprender a segmentar las regiones grandes y frecuentes de una escena como el cielo y los edificios, su desempeño en las clases pequeñas y poco frecuentes depende de cuántos ejemplos tiene de ellas y esto se vio en la correlación de 0.71 entre la frecuencia de cada clase y su IoU.

Se comprobó la importancia de diagnosticar el tipo de error antes de aplicar una técnica de mejora, al identificar el sobreajuste se eligió data augmentation y esto mejoró el mIoU de prueba de 0.144 a 0.166 sin cambiar la arquitectura, el siguiente paso es reducir el sobreajuste (overfit) y compensar el desbalance de clases con las técnicas de la sección de trabajo futuro.

Referencias

[1] “Image segmentation,” TensorFlow Core Tutorials. https://www.tensorflow.org/tutorials/images/segmentation

[2] J. Long, E. Shelhamer, and T. Darrell, “Fully convolutional networks for semantic segmentation,” in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2015, pp. 3431–3440.

[3] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in Proc. Med. Image Comput. Comput.-Assist. Interv. (MICCAI), 2015, pp. 234–241. https://arxiv.org/abs/1505.04597

[4] B. Zhou, H. Zhao, X. Puig, S. Fidler, A. Barriuso, and A. Torralba, “Scene parsing through ADE20K dataset,” in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017, pp. 633–641. http://sceneparsing.csail.mit.edu/

[5] “Development kit for MIT Scene Parsing Benchmark,” MIT CSAIL, GitHub repository. https://github.com/CSAILVision/sceneparsing

[6] “How To Choose The Right Test Options When Evaluating Machine Learning Algorithms,” Machine Learning Mastery.

https://machinelearningmastery.com/how-to-choose-the-right-test-options-when-evaluating-machine-learning-algorithms/

[7] "tf.data: Build TensorFlow input pipelines," TensorFlow Core Guide. https://www.tensorflow.org/guide/data

[8] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in Proc. Int. Conf. Mach. Learn. (ICML), 2015, pp. 448–456. https://arxiv.org/abs/1502.03167

[9] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017, pp. 1125–1134.

[10] "Segmentation metrics: MeanIoU," Keras documentation. https://keras.io/api/metrics/segmentation_metrics/

[11] "Mixed precision," TensorFlow Core Guide. https://www.tensorflow.org/guide/mixed_precision

[12] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, arXiv:1412.6980. https://arxiv.org/abs/1412.6980

[13] "ML | Underfitting and Overfitting," GeeksforGeeks. https://www.geeksforgeeks.org/underfitting-and-overfitting-in-machine-learning/